PPI 8255 vs LATCHES
Comportamento em Leitura de I/O no Z80

Autor : Marcos Roberto Braga
Projeto : Z80-EMU – Emulador de Z80 em PIC18F6722
Data : 2025
Licença : Material didático – livre para uso educacional

SUMÁRIO

1. Introdução
2. O que é a PPI 8255
3. O que é um Latch (74LS273)
4. A diferença fundamental: latch de entrada
5. O problema com IND e INDR
6. Comportamento em hardware real
7. Comportamento em simulação
8. A solução: latches discretos
9. Lição para engenheiros

1. INTRODUÇÃO

Ao testar as instruções IND e INDR do Z80 em um hardware que usa
PPI 8255 para entrada de dados, foi observado um comportamento
inesperado: apenas o primeiro byte era lido corretamente. Os

bytes seguintes vinham vazios ou indefinidos.

Após investigação, descobriu-se que a causa não era um bug no código, mas uma CARACTERÍSTICA do hardware: a PPI 8255 no Modo 0 não possui latch de entrada.

Este documento explica a diferença entre a PPI 8255 e os latches discretos (74LS273), e por que isso afeta o comportamento das instruções de bloco do Z80.

---

2. O QUE É A PPI 8255

A PPI 8255 (Programmable Peripheral Interface) é um circuito integrado da Intel, muito usado em sistemas Z80 para interface de I/O paralelo. Ela possui:

- 3 portas de 8 bits (A, B, C)
- 3 modos de operação (Modo 0, 1, 2)
- Configuração programável (entrada ou saída por porta)
- 24 pinos de I/O

No MODO 0 (I/O simples):

- As portas podem ser configuradas como entrada ou saída
- Não há latch de entrada nas portas configuradas como entrada
- A leitura é feita DIRETAMENTE do pino

Isso significa que, se o pino estiver em ALTA IMPEDÂNCIA (Hi-Z), a leitura retorna um valor indefinido.

3. O QUE É UM LATCH (74LS273)

O 74LS273 é um latch octal (8 bits) com saída tri-state,
muito usado para armazenar dados de entrada de forma estável.

Ele possui:
- 8 entradas de dados (D0-D7)
- 8 saídas de dados (Q0-Q7)
- Pino LE (Latch Enable) — controla a captura
- Pino OE (Output Enable) — controla a saída

Comportamento:
- Quando LE está ativo, o latch captura o valor das entradas
- Quando LE volta a inativo, o latch MANTÉM o valor capturado
- A saída só é ativa quando OE está ativo

Isso significa que o valor fica ESTÁVEL até a próxima captura —
independente de quantas vezes o Z80 o leia.

4. A DIFERENÇA FUNDAMENTAL: LATCH DE ENTRADA

PPI 8255 (Modo 0)	LATCH (74LS273)
Sem latch de entrada Lê o pino diretamente Valor pode flutuar (Hi-Z)	Com latch de entrada Lê o valor capturado Valor permanece estável

Cada leitura = valor atual	Cada leitura = mesmo valor
Depende do dispositivo	Independente do dispositivo

Essa diferença é CRUCIAL para instruções que fazem MÚLTIPLAS
leituras consecutivas, como IND e INDR.

---

5. O PROBLEMA COM IND E INDR

As instruções IND e INDR do Z80 fazem leituras consecutivas
da porta de I/O apontada por C, gravando o resultado na memória
apontada por HL, decrementando HL e B a cada iteração.

INDR (ED BAh):
  (HL) ← (C); HL--; B--; repete até B=0

Em hardware com PPI 8255 Modo 0:
  1ª leitura → valor correto (o pino está estável)
  2ª leitura → valor indefinido (o pino flutua em Hi-Z)
  3ª leitura → valor indefinido
  ...
  8ª leitura → valor indefinido

Resultado: apenas 1 ou 2 bytes são gravados corretamente.

Em hardware com latch 74LS273:
  1ª leitura → valor estável (latch mantém o valor)
  2ª leitura → mesmo valor
  3ª leitura → mesmo valor

...
8ª leitura → mesmo valor

Resultado: os 8 bytes são gravados corretamente.

---

6. COMPORTAMENTO EM HARDWARE REAL

Em hardware real, o comportamento é o mesmo:

COM PPI 8255 (Modo 0, sem latch externo):
- A leitura depende do dispositivo externo manter o dado estável
- Se o dispositivo flutua entre leituras, o valor é indefinido
- Em sistemas reais, isso causa bugs intermitentes

COM LATCH 74LS273:
- O dado é capturado uma vez e mantido estável
- A leitura é sempre determinística
- É o padrão usado em sistemas críticos

Por isso, sistemas profissionais NÃO usam PPI 8255 para entrada de dados em aplicações que exigem estabilidade. Usam latches ou buffers com controle de saída.

---

7. COMPORTAMENTO EM SIMULAÇÃO

Em simuladores como o Proteus, o comportamento é similar:

COM PPI 8255:

- O modelo da PPI não mantém o dado estável entre leituras
- Apenas a 1ª leitura retorna o valor correto
- As leituras seguintes retornam vazio ou indefinido

COM LATCH 74LS273:

- O modelo do latch mantém o dado estável
- Todas as leituras retornam o mesmo valor
- Fiel ao comportamento de hardware real

IMPORTANTE: a limitação da PPI no simulador REFLETE a limitação do hardware real. Não é um bug do simulador — é uma característica do componente.

---

8. A SOLUÇÃO: LATCHES DISCRETOS

Para testar instruções que dependem de estabilidade (IND, INDR, INI, INIR), recomenda-se usar LATCHES DISCRETOS em vez de PPI.

Hardware sugerido:

- 74LS273 — latch de 8 bits para entrada
- 74LS244 — buffer tri-state para saída
- 74LS138 — decodificador de endereço

Mapeamento de exemplo:

Porta A (01h) — SAÍDA (74LS273 → LEDs)
Porta B (02h) — ENTRADA (74LS244 ← DIP switches)
Porta C (10h) — SAÍDA (74LS273 → LEDs)

Com esse hardware, as instruções IND e INDR funcionam
corretamente — como em sistemas reais.

---

9. LIÇÃO PARA ENGENHEIROS

Esta experiência ensina três lições fundamentais:

1. NEM TODO BUG É DO CÓDIGO
   O comportamento inesperado da PPI não era um bug do emulador
   ou do ASM. Era uma característica do hardware.

2. O HARDWARE INFLUENCIA O SOFTWARE
   A mesma instrução (INDR) funciona com latch e falha com PPI.
   O comportamento depende do hardware — não do código.

3. SIMULADORES TÊM LIMITES
   O Proteus modela a PPI com fidelidade limitada. Isso não é
   um defeito — é uma característica. Saber reconhecer esses
   limites é essencial.

CONCLUSÃO:

  "A PPI 8255 no Modo 0 não tem latch de entrada. Se o
   dispositivo externo não mantém o dado estável, a leitura
   é indefinida. Os latches resolvem isso — e é por isso que
   sistemas profissionais os usam."

REFERÊNCIAS

- Intel 8255A Datasheet — Programmable Peripheral Interface
- Texas Instruments 74LS273 Datasheet — Octal D-Type Latch
- Zilog Z80 CPU User Manual — Input/Output Instructions
- Z80-EMU — Repositório do projeto (GitHub)

FIM DO DOCUMENTO
